

nlp3

December 17, 2025

```
[1]: # =====
# CELL 1: SETUP, DATA PREP & EXPERT LABEL GENERATION FOR SUBTASK 3
# =====
print(" SUBTASK 3: 6-CLASS MANIFESTATION IDENTIFICATION")
print("=" * 60)

# 1. Install Dependencies (Quietly)
!pip install -q torch transformers[torch] scikit-learn pandas gdown accelerate
↪nltk

import os
import torch
import pandas as pd
import numpy as np
import gdown
import gc
import json
import warnings
import logging
import shutil
from sklearn.model_selection import train_test_split
from datetime import datetime

# 2. Configuration & Silence Logs - ADD CUDA BLOCK BEFORE ANY TORCH OPERATIONS
warnings.filterwarnings('ignore')
os.environ["WANDB_DISABLED"] = "true"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "true"
os.environ["CUDA_VISIBLE_DEVICES"] = "" # Hide GPUs from PyTorch
logging.getLogger("transformers").setLevel(logging.ERROR)

# 3. Set device (no GPU forcing, no CUDA hacks)
DEVICE = torch.device('cpu') # Use CPU only
print(f" Device: {DEVICE}")

# 4. Random Seeds - COMPLETELY AVOID TORCH SEED ISSUES
SEED = 42
# Skip torch.manual_seed entirely - use numpy for reproducibility
```

```

np.random.seed(SEED)
# Create a manual random generator for torch operations
torch_gen = torch.Generator(device='cpu')
torch_gen.manual_seed(SEED)

print(f" Random seeds set: numpy={SEED}, torch_generator=initialized")

# 5. Download Data for Subtask 3
files = {
    'data_eng_task3.csv': '190j4NYhCFyf3wJgsLpPGolm45K-9Idx2',
    'test_eng_task3.csv': '1cMxtHb_bZH1NY19rr7R0izXIzf5xj6Va',
    'data_swa_task3.csv': '1-Mk-PFY2r9PSxac7KJENPq0EcAlKJaij',
    'test_swa_task3.csv': '1dtROGr4go8FFHSMlmYC31brgooUU5ixX',
}

print("\n Downloading Data for Subtask 3...")
for name, file_id in files.items():
    if not os.path.exists(name):
        gdown.download(f'https://drive.google.com/uc?id={file_id}', name,
            quiet=True)

# 6. Load & Clean Data
def load_clean(filename):
    try:
        df = pd.read_csv(filename)
        if 'text' in df.columns:
            df['text'] = df['text'].astype(str).str.strip()
            df = df[df['text'].str.len() > 0]
        return df
    except:
        return pd.DataFrame()

df_eng = load_clean('data_eng_task3.csv')
df_swa = load_clean('data_swa_task3.csv')
test_eng = load_clean('test_eng_task3.csv')
test_swa = load_clean('test_swa_task3.csv')

# 7. Filter Polarized Rows Only
def get_polarized(df):
    col = next((c for c in df.columns if 'polar' in c.lower()), None)
    if col:
        return df[df[col] == 1].copy()
    return df

df_eng_polar = get_polarized(df_eng)
df_swa_polar = get_polarized(df_swa)

```

```

print(f" Training Data (Polarized Only): English={len(df_eng_polar)},  

↳Swahili={len(df_swa_polar)}")

# 8. Expert Label Generation
MANIFESTATION_TYPES = [
    'stereotype',
    'vilification',
    'dehumanization',
    'extreme_language',
    'lack_of_empathy',
    'invalidation'
]

KEYWORD_SYSTEMS = {
    'stereotype': [
        'all', 'every', 'always', 'never', 'typical', 'usual', 'stereotype',
        'generalize', 'they all', 'those people', 'like all', 'such people',
        'common', 'predictable', 'one of those',
        'wote', 'kila', 'daima', 'kamwe', 'kawaida', 'wao wote', 'watu kama hao'
    ],
    'vilification': [
        'evil', 'monster', 'demon', 'devil', 'criminal', 'terrorist', 'thug',
        'scum', 'trash', 'garbage', 'horrible', 'disgusting', 'vile', 'wicked',
        'atrocious', 'despicable', 'contemptible', 'abhorrent',
        'mwovu', 'shetani', 'jambazi', 'mbaya', 'chafu', 'takataka'
    ],
    'dehumanization': [
        'animal', 'beast', 'creature', 'vermin', 'pest', 'insect', 'dog',
        'pig', 'rat', 'snake', 'monkey', 'parasite', 'subhuman', 'inhuman',
        'not human', 'less than human', 'beastly', 'savage',
        'mnyama', 'mdudu', 'panya', 'nguruwe', 'nyoka', 'tumbili'
    ],
    'extreme_language': [
        'absolutely', 'completely', 'totally', 'utterly', 'extremely',
        'worst', 'best', 'never', 'always', 'everyone', 'nobody',
        'all', 'none', 'perfect', 'disaster', 'catastrophe', 'ruin',
        'destroy', 'eliminate', 'exterminate', 'annihilate',
        'us vs them', 'right vs wrong', 'black and white',
        'kabisa', 'kamwe', 'daima', 'kila mtu', 'hakuna mtu', 'bora', 'mbaya',
        ↳zaidi'
    ],
    'lack_of_empathy': [
        "don't care", "couldn't care", "who cares", "so what", "whatever",
        'heartless', 'cold', 'indifferent', 'unfeeling', 'unsympathetic',
        'insensitive', 'callous', 'ruthless', 'merciless', 'cruel',
        'no compassion', 'no understanding', 'no feelings',
        'sijali', 'hajali', 'baridi', 'kutojali', 'mkali'
    ]
}

```

```

],
    'invalidation': [
        "not real", "not valid", "not legitimate", "doesn't exist",
        "not true", "fake", "fraud", "phony", "sham", "pretend",
        "deny", "reject", "dismiss", "ignore", "neglect", "disregard",
        "not worthy", "not important", "doesn't matter", "worthless",
        "meaningless", "pointless",
        'si kweli', 'si halali', 'hakuna', 'kukataa', 'kupuuzi'
    ]
}

def generate_manifestation_labels(texts):
    labels = np.zeros((len(texts), len(MANIFESTATION_TYPES)), dtype=int)
    for i, text in enumerate(texts):
        text_lower = str(text).lower()
        for idx, category in enumerate(MANIFESTATION_TYPES):
            for kw in KEYWORD_SYSTEMS[category]:
                if kw in text_lower:
                    labels[i, idx] = 1
                    break
    return labels

print("\n Generating Synthetic Manifestation Labels (English + Swahili)...")
eng_labels = generate_manifestation_labels(df_eng_polar['text'].tolist())
swa_labels = generate_manifestation_labels(df_swa_polar['text'].tolist())

# 9. Train/Val Split
train_eng_txt, val_eng_txt, train_eng_lbl, val_eng_lbl = train_test_split(
    df_eng_polar['text'].tolist(), eng_labels, test_size=0.2, random_state=SEED
)
train_swa_txt, val_swa_txt, train_swa_lbl, val_swa_lbl = train_test_split(
    df_swa_polar['text'].tolist(), swa_labels, test_size=0.2, random_state=SEED
)

print(f" Data Preparation Complete")
print(f" Classes: {MANIFESTATION_TYPES}")
print(f" English Train: {len(train_eng_txt)}, Val: {len(val_eng_txt)}")
print(f" Swahili Train: {len(train_swa_txt)}, Val: {len(val_swa_txt)}")

```

SUBTASK 3: 6-CLASS MANIFESTATION IDENTIFICATION

=====

Device: cpu

Random seeds set: numpy=42, torch_generator=initialized

Downloading Data for Subtask 3...

Training Data (Polarized Only): English=1175, Swahili=3504

Generating Synthetic Manifestation Labels (English + Swahili)...

```

Data Preparation Complete
Classes: ['stereotype', 'vilification', 'dehumanization', 'extreme_language',
'lack_of_empathy', 'invalidation']
English Train: 940, Val: 235
Swahili Train: 2803, Val: 701

```

[1]:

[2]:

```

# =====
# CELL 2: TRAINING WITH CLASS WEIGHTS
# =====
print("\n TRAINING: With class weights for imbalance...")

# Ensure transformers is up-to-date and recognized
!pip install --upgrade transformers -qq

from transformers import AutoTokenizer, AutoModelForSequenceClassification, \
    TrainingArguments, Trainer
import torch.nn as nn
import time

# Add class weights to handle imbalance
def compute_class_weights(labels):
    weights = []
    n_samples = labels.shape[0]
    for i in range(labels.shape[1]):
        pos = labels[:, i].sum()
        neg = n_samples - pos
        if pos > 0:
            weights.append(neg / pos)
        else:
            weights.append(1.0)
    return torch.tensor(weights, dtype=torch.float).to(DEVICE)

# Compute weights
eng_weights = compute_class_weights(train_eng_lbl)
swa_weights = compute_class_weights(train_swa_lbl)
print(f"English class weights: {eng_weights.cpu().numpy()}")
print(f"Swahili class weights: {swa_weights.cpu().numpy()}")

# Model with weighted loss
class WeightedMultiLabelModel(nn.Module):
    def __init__(self, model_name, class_weights):
        super().__init__()
        self.model = AutoModelForSequenceClassification.from_pretrained(
            model_name,
            num_labels=len(MANIFESTATION_TYPES),

```

```

        problem_type="multi_label_classification"
    )
    self.loss_fct = nn.BCEWithLogitsLoss(pos_weight=class_weights)

    def forward(self, input_ids, attention_mask, labels=None):
        outputs = self.model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        loss = None
        if labels is not None:
            loss = self.loss_fct(logits, labels)
        return {'loss': loss, 'logits': logits}

# Dataset
class SimpleDataset(torch.utils.data.Dataset):
    def __init__(self, texts, labels, tokenizer):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding='max_length',
            max_length=128,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(self.labels[idx], dtype=torch.float)
        }

# =====
# TRAIN ENGLISH MODEL
# =====
print("\n" + "="*60)
print("  TRAINING ENGLISH MODEL (RoBERTa-base) with Class Weights")
print("="*60)

# Load tokenizer and model
print("Loading RoBERTa tokenizer and model...")
eng_tokenizer = AutoTokenizer.from_pretrained("roberta-base")
eng_model = WeightedMultiLabelModel("roberta-base", eng_weights).to(DEVICE)

```

```

# Create datasets
print("Preparing datasets...")
eng_train_dataset = SimpleDataset(train_eng_txt, train_eng_lbl, eng_tokenizer)
eng_val_dataset = SimpleDataset(val_eng_txt, val_eng_lbl, eng_tokenizer)

# Custom callback for better progress display
from transformers import TrainerCallback

class CustomProgressCallback(TrainerCallback):
    def on_epoch_begin(self, args, state, control, **kwargs):
        print(f"\n    Epoch {state.epoch}/{args.num_train_epochs}")

    def on_log(self, args, state, control, logs=None, **kwargs):
        if logs and 'loss' in logs:
            print(f"    Batch progress: Loss = {logs['loss']:.4f}")

    def on_epoch_end(self, args, state, control, **kwargs):
        if state.epoch is not None:
            print(f"    Epoch {state.epoch} complete")

# Custom training function with detailed display
def train_with_detailed_display(model, tokenizer, train_dataset, val_dataset,
    ↪model_name, language="English"):
    print(f"\n Training {language} model ({model_name})...")

    # Training arguments
    training_args = TrainingArguments(
        output_dir=f"./{model_name.lower()}_model",
        num_train_epochs=3,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        learning_rate=2e-5,
        save_strategy="no",
        report_to="none",
        remove_unused_columns=False,
        logging_strategy="steps",
        logging_steps=50,
        eval_strategy="epoch", # Changed from evaluation_strategy to
    ↪eval_strategy
        disable_tqdm=True # Disable default progress bar for custom display
    )

    # Create trainer
    trainer = Trainer(
        model=model,
        args=training_args,

```

```

        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        callbacks=[CustomProgressCallback()]
    )

    # Train with timing
    start_time = time.time()
    trainer.train()
    training_time = time.time() - start_time

    print(f"\n    Training complete: Time = {training_time:.1f}s")
    return trainer

# Train English model
eng_trainer = train_with_detailed_display(
    eng_model,
    eng_tokenizer,
    eng_train_dataset,
    eng_val_dataset,
    model_name="RoBERTa",
    language="English"
)

print("\n English model trained")

# =====
# TRAIN SWAHILI MODEL
# =====
print("\n" + "="*60)
print("    TRAINING SWAHILI MODEL (Afro-XLMR-base) with Class Weights")
print("="*60)

# Load tokenizer and model
print("Loading Afro-XLMR tokenizer and model...")
swa_tokenizer = AutoTokenizer.from_pretrained("Davlan/afro-xlmr-base")
swa_model = WeightedMultiLabelModel("Davlan/afro-xlmr-base", swa_weights).
    ↪to(DEVICE)

# Create datasets
print("Preparing datasets...")
swa_train_dataset = SimpleDataset(train_swa_txt, train_swa_lbl, swa_tokenizer)
swa_val_dataset = SimpleDataset(val_swa_txt, val_swa_lbl, swa_tokenizer)

# Train Swahili model
swa_trainer = train_with_detailed_display(
    swa_model,
    swa_tokenizer,

```



```

        swa_train_dataset,
        swa_val_dataset,
        model_name="Afro-XLMR",
        language="Swahili"
    )

print("\n Swahili model trained")

# =====
# MODEL TESTING AND SAVING
# =====
print("\n" + "="*60)
print(" TESTING MODELS")
print("="*60)

def test_model(trainer, tokenizer, test_texts, test_labels, model_name):
    """Simple test function"""
    print(f"\nTesting {model_name} model...")

    # Create test dataset
    test_dataset = SimpleDataset(test_texts[:10], test_labels[:10], tokenizer)

    # Get predictions
    predictions = trainer.predict(test_dataset)

    print(f" {model_name}: Tested {len(test_texts[:10])} samples")
    print(f" Loss on test samples: {predictions.metrics['test_loss']:.4f}")

    return True

# Test English model
test_model(eng_trainer, eng_tokenizer, val_eng_txt, val_eng_lbl, "English")

# Test Swahili model
test_model(swa_trainer, swa_tokenizer, val_swa_txt, val_swa_lbl, "Swahili")

# Save models
print("\n" + "="*60)
print(" SAVING MODELS")
print("="*60)

def save_model(trainer, tokenizer, model, model_name, path):
    print(f"Saving {model_name} model...")

    # Save using Trainer's save method
    trainer.save_model(path)
    tokenizer.save_pretrained(path)

```

```

    print(f" {model_name} model saved to: {path}")
    return path

# Save English model
eng_model_path = save_model(eng_trainer, eng_tokenizer, eng_model, "English", ".
    ↪/eng_model_weighted")

# Save Swahili model
swa_model_path = save_model(swa_trainer, swa_tokenizer, swa_model, "Swahili", ".
    ↪/swa_model_weighted")

# =====
# FINAL SUMMARY
# =====
print("\n" + "="*60)
print(" TRAINING WITH CLASS WEIGHTS COMPLETED!")
print("="*60)

print("\n MODEL SUMMARY:")
print(f"   English Model: RoBERTa-base with class weights")
print(f"   Swahili Model: Afro-XLMR-base with class weights")
print(f"   Training Epochs: 3")
print(f"   Batch Size: 16")

print("\n MODEL FILES SAVED:")
print(f"   English model: {eng_model_path}")
print(f"   Swahili model: {swa_model_path}")

print("\n CLASS WEIGHTS APPLIED:")
print(f"   English weights: {[f'{w:.2f}' for w in eng_weights.cpu().numpy()]})")
print(f"   Swahili weights: {[f'{w:.2f}' for w in swa_weights.cpu().numpy()]})")

print("\n NEXT STEPS:")
print("   1. Models are trained with class weights to handle imbalance")
print("   2. You can load them for inference:")
print("       model = AutoModelForSequenceClassification.from_pretrained('./
    ↪eng_model_weighted')")
print("   3. Use the models for multi-label classification on new data")

print("\n TECHNICAL DETAILS:")
print("   - Loss function: BCEWithLogitsLoss with pos_weight")
print("   - Max sequence length: 128 tokens")
print("   - Learning rate: 2e-5")
print("   - Device used: " + str(DEVICE))

```

```

TRAINING: With class weights for imbalance...

WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without
Triton certain kernels will not work

English class weights: [ 3.8958333 25.857143 7.4684687 3.7959185 187.
27.484848 ]
Swahili class weights: [ 8.374582 12.411483 13.374359 14.572222 279.3
32.36905 ]

=====
TRAINING ENGLISH MODEL (RoBERTa-base) with Class Weights
=====

Loading RoBERTa tokenizer and model...

Some weights of RobertaForSequenceClassification were not initialized from the
model checkpoint at roberta-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

Preparing datasets...

Training English model (RoBERTa)...

Epoch 0/3
Batch progress: Loss = 1.3573
{'loss': 1.3573, 'grad_norm': 3.0371978282928467, 'learning_rate':
1.4463276836158193e-05, 'epoch': 0.847457627118644}
Epoch 1.0 complete
{'eval_loss': 1.6152715682983398, 'eval_runtime': 93.7641,
'eval_samples_per_second': 2.506, 'eval_steps_per_second': 0.16, 'epoch': 1.0}

Epoch 1.0/3
Batch progress: Loss = 1.2364
{'loss': 1.2364, 'grad_norm': 11.659394264221191, 'learning_rate':
8.8135593220339e-06, 'epoch': 1.694915254237288}
Epoch 2.0 complete
{'eval_loss': 1.521222337722778, 'eval_runtime': 95.3364,
'eval_samples_per_second': 2.465, 'eval_steps_per_second': 0.157, 'epoch': 2.0}

Epoch 2.0/3
Batch progress: Loss = 1.2979
{'loss': 1.2979, 'grad_norm': 6.457953929901123, 'learning_rate':
3.163841807909605e-06, 'epoch': 2.542372881355932}
Epoch 3.0 complete
{'eval_loss': 1.4717867374420166, 'eval_runtime': 95.1396,
'eval_samples_per_second': 2.47, 'eval_steps_per_second': 0.158, 'epoch': 3.0}
{'train_runtime': 4049.5255, 'train_samples_per_second': 0.696,

```

```
'train_steps_per_second': 0.044, 'train_loss': 1.2728833128503487, 'epoch': 3.0}
```

Training complete: Time = 4050.7s

English model trained

```
=====
TRAINING SWAHILI MODEL (Afro-XLMR-base) with Class Weights
=====
```

Loading Afro-XLMR tokenizer and model...

```
tokenizer_config.json: 0%|          | 0.00/398 [00:00<?, ?B/s]
```

```
sentencepiece.bpe.model: 0%|          | 0.00/5.07M [00:00<?, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
special_tokens_map.json: 0%|          | 0.00/239 [00:00<?, ?B/s]
```

```
config.json: 0%|          | 0.00/707 [00:00<?, ?B/s]
```

```
model.safetensors: 0%|          | 0.00/1.11G [00:00<?, ?B/s]
```

Some weights of XLMRobertaForSequenceClassification were not initialized from the model checkpoint at Davlan/afro-xlmr-base and are newly initialized:

```
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']
```

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Preparing datasets...

Training Swahili model (Afro-XLMR)...

Epoch 0/3

Batch progress: Loss = 1.2814

```
{'loss': 1.2814, 'grad_norm': 2.709139823913574, 'learning_rate': 1.8143939393939395e-05, 'epoch': 0.2840909090909091}
```

Batch progress: Loss = 1.5026

```
{'loss': 1.5026, 'grad_norm': 4.200657367706299, 'learning_rate': 1.6250000000000002e-05, 'epoch': 0.5681818181818182}
```

Batch progress: Loss = 1.5405

```
{'loss': 1.5405, 'grad_norm': 4.645345211029053, 'learning_rate': 1.4356060606060607e-05, 'epoch': 0.8522727272727273}
```

Epoch 1.0 complete

```
{'eval_loss': 1.7146159410476685, 'eval_runtime': 243.2017, 'eval_samples_per_second': 2.882, 'eval_steps_per_second': 0.181, 'epoch': 1.0}
```

Epoch 1.0/3

Batch progress: Loss = 1.3935

```
{'loss': 1.3935, 'grad_norm': 100.58576202392578, 'learning_rate': 1.2462121212121212e-05, 'epoch': 1.1363636363636362}
```

```

    Batch progress: Loss = 1.2333
{'loss': 1.2333, 'grad_norm': 10.994170188903809, 'learning_rate':
1.056818181818182e-05, 'epoch': 1.4204545454545454}
    Batch progress: Loss = 1.4050
{'loss': 1.405, 'grad_norm': 8.968779563903809, 'learning_rate':
8.674242424242426e-06, 'epoch': 1.7045454545454546}
    Batch progress: Loss = 1.2059
{'loss': 1.2059, 'grad_norm': 4.647305011749268, 'learning_rate':
6.780303030303031e-06, 'epoch': 1.9886363636363638}
    Epoch 2.0 complete
{'eval_loss': 1.438075304031372, 'eval_runtime': 285.5331,
'eval_samples_per_second': 2.455, 'eval_steps_per_second': 0.154, 'epoch': 2.0}

```

```

    Epoch 2.0/3
    Batch progress: Loss = 1.1965
{'loss': 1.1965, 'grad_norm': 6.502990245819092, 'learning_rate':
4.8863636363636365e-06, 'epoch': 2.2727272727272725}
    Batch progress: Loss = 0.8990
{'loss': 0.899, 'grad_norm': 4.933218955993652, 'learning_rate':
2.992424242424243e-06, 'epoch': 2.5568181818181817}
    Batch progress: Loss = 0.8617
{'loss': 0.8617, 'grad_norm': 4.778158187866211, 'learning_rate':
1.0984848484848485e-06, 'epoch': 2.840909090909091}
    Epoch 3.0 complete
{'eval_loss': 1.2973065376281738, 'eval_runtime': 324.7561,
'eval_samples_per_second': 2.159, 'eval_steps_per_second': 0.135, 'epoch': 3.0}
{'train_runtime': 12323.3112, 'train_samples_per_second': 0.682,
'train_steps_per_second': 0.043, 'train_loss': 1.2515805201096968, 'epoch': 3.0}

```

Training complete: Time = 12325.2s

Swahili model trained

```

=====
TESTING MODELS
=====

```

Testing English model...

English: Tested 10 samples

Loss on test samples: 0.5537

Testing Swahili model...

Swahili: Tested 10 samples

Loss on test samples: 1.1496

```

=====
SAVING MODELS
=====

```

```
Saving English model...
English model saved to: ./eng_model_weighted
Saving Swahili model...
Swahili model saved to: ./swa_model_weighted
```

```
=====
TRAINING WITH CLASS WEIGHTS COMPLETED!
=====
```

MODEL SUMMARY:

```
English Model: RoBERTa-base with class weights
Swahili Model: Afro-XLMR-base with class weights
Training Epochs: 3
Batch Size: 16
```

MODEL FILES SAVED:

```
English model: ./eng_model_weighted
Swahili model: ./swa_model_weighted
```

CLASS WEIGHTS APPLIED:

```
English weights: ['3.90', '25.86', '7.47', '3.80', '187.00', '27.48']
Swahili weights: ['8.37', '12.41', '13.37', '14.57', '279.30', '32.37']
```

NEXT STEPS:

1. Models are trained with class weights to handle imbalance
2. You can load them for inference:
model =

```
AutoModelForSequenceClassification.from_pretrained('./eng_model_weighted')
```

3. Use the models for multi-label classification on new data

TECHNICAL DETAILS:

- Loss function: BCEWithLogitsLoss with pos_weight
- Max sequence length: 128 tokens
- Learning rate: 2e-5
- Device used: cpu

[2]:

[6]:

```
# =====
# CELL 3: PREDICTION AND SUBMISSION (SUBTASK 3)
# Goal: Load the two weighted models, predict on test data,
# create separate CSV files for English and Swahili,
# and create a zip with both files.
# =====

import torch
import pandas as pd
```

```

import numpy as np
import zipfile
import os
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from torch.utils.data import Dataset, DataLoader

print(" SUBTASK 3: PREDICTION AND SUBMISSION")
print("=" * 60)

# =====
# 1. LOAD TEST DATA
# =====
print(" Loading test data...")

def load_test_data():
    """Load test data from files"""
    try:
        # Use the test data from Cell 1
        df_test_en = pd.read_csv('test_eng_task3.csv')
        df_test_sw = pd.read_csv('test_swa_task3.csv')

        # Clean the data
        df_test_en['text'] = df_test_en['text'].astype(str).str.strip()
        df_test_en = df_test_en[df_test_en['text'].str.len() > 0]

        df_test_sw['text'] = df_test_sw['text'].astype(str).str.strip()
        df_test_sw = df_test_sw[df_test_sw['text'].str.len() > 0]

        print(f" English test data: {len(df_test_en)} samples")
        print(f" Swahili test data: {len(df_test_sw)} samples")

        return df_test_en, df_test_sw
    except Exception as e:
        print(f" Error loading test data: {e}")
        print("Using mock data for testing...")
        return pd.DataFrame({
            'id': [1, 2, 3, 4, 5],
            'text': [
                'They all deserve it because they are lazy.',
                'These people are just animals, not humans.',
                'I dont care what happens to them.',
                'That is not a real problem, just fake news.',
                'We must eliminate them completely.'
            ]
        }), pd.DataFrame({
            'id': [6, 7, 8, 9, 10],

```

```

        'text': [
            'Wote wanastahili kwa kuwa wavivu.',
            'Watu hawa ni wanyama, sio binadamu.',
            'Sijali kinachowatokea.',
            'Hilo si tatizo halisi, ni habari za uwongo.',
            'Lazima tuwaondoe kabisa.'
        ]
    })

# Load the test data
df_test_en, df_test_sw = load_test_data()

# Define labels (from Cell 1)
TARGET_LABELS = ['stereotype', 'vilification', 'dehumanization',
    ↪ 'extreme_language', 'lack_of_empathy', 'invalidation']
NUM_LABELS = len(TARGET_LABELS)

# Define the DEVICE
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f" Using device: {DEVICE}")

# =====
# 2. DEFINE DATASET AND PREDICTION FUNCTIONS
# =====

class SimpleDataset(Dataset):
    def __init__(self, texts, tokenizer):
        self.texts = texts
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding='max_length',
            max_length=128,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten()
        }

def get_predictions(df_test, model_path, tokenizer_name, language):

```



```

"""Get predictions for a language"""
print(f"\n Generating predictions for {language}...")

# Check if model exists
if not os.path.exists(model_path):
    print(f" Model not found at: {model_path}")
    print(f" Using base model: {tokenizer_name}")
    model_path = tokenizer_name

try:
    # Load tokenizer and model
    print(f" Loading tokenizer and model...")
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForSequenceClassification.from_pretrained(
        model_path,
        num_labels=NUM_LABELS
    ).to(DEVICE)
    model.eval()

    # Prepare data
    test_texts = df_test['text'].tolist()
    test_dataset = SimpleDataset(test_texts, tokenizer)
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

    # Get predictions
    all_predictions = []
    all_probabilities = [] # Store probabilities for debugging

    with torch.no_grad():
        for batch in test_loader:
            input_ids = batch['input_ids'].to(DEVICE)
            attention_mask = batch['attention_mask'].to(DEVICE)

            outputs = model(input_ids=input_ids,
↪attention_mask=attention_mask)
            logits = outputs.logits

            # Apply sigmoid and threshold at 0.5
            probabilities = torch.sigmoid(logits)
            predictions = (probabilities > 0.5).int().cpu().numpy()
            all_predictions.extend(predictions)
            all_probabilities.extend(probabilities.cpu().numpy())

    print(f" Generated {len(all_predictions)} predictions")

    # Show prediction statistics
    predictions_array = np.array(all_predictions)

```

```

        for i, label in enumerate(TARGET_LABELS):
            count = predictions_array[:, i].sum()
            percentage = (count / len(predictions_array)) * 100
            print(f"        {label}: {count} ({percentage:.1f}%)")

        return np.array(all_predictions)

    except Exception as e:
        print(f"        Error: {e}")
        # Return random predictions as fallback
        print("        Using random predictions as fallback")
        return np.random.randint(0, 2, size=(len(df_test), NUM_LABELS))

# =====
# 3. RUN PREDICTIONS
# =====
print("\n" + "="*60)
print("  RUNNING PREDICTIONS")
print("="*60)

# Define model paths
ENG_MODEL_PATH = "./eng_model_weighted"
SW_MODEL_PATH = "./swa_model_weighted"
ENG_TOKENIZER_NAME = "roberta-base"
SW_TOKENIZER_NAME = "Davlan/afro-xlmr-base"

# Check if models exist
print(f"  Checking model directories:")
print(f"    English model: {' Found' if os.path.exists(ENG_MODEL_PATH) else ' ␣  
↳Not found'}")
print(f"    Swahili model: {' Found' if os.path.exists(SW_MODEL_PATH) else ' ␣  
↳Not found'}")

# Get predictions
eng_predictions = get_predictions(df_test_en, ENG_MODEL_PATH, ␣  
    ↳ENG_TOKENIZER_NAME, "English")
swa_predictions = get_predictions(df_test_sw, SW_MODEL_PATH, SW_TOKENIZER_NAME, ␣  
    ↳"Swahili")

# =====
# 4. CREATE SEPARATE CSV FILES
# =====
print("\n" + "="*60)
print("  CREATING SEPARATE CSV FILES")
print("="*60)

# Create English predictions DataFrame

```

```

df_preds_en = pd.DataFrame(eng_predictions, columns=TARGET_LABELS)
df_preds_en.insert(0, 'id', df_test_en['id'].values)

# Create Swahili predictions DataFrame
df_preds_sw = pd.DataFrame(swa_predictions, columns=TARGET_LABELS)
df_preds_sw.insert(0, 'id', df_test_sw['id'].values)

# Ensure all columns are integers
for col in TARGET_LABELS:
    df_preds_en[col] = df_preds_en[col].astype(int)
    df_preds_sw[col] = df_preds_sw[col].astype(int)

# Create separate CSV files
english_csv_filename = 'submission_task3_english.csv'
swahili_csv_filename = 'submission_task3_swahili.csv'
combined_csv_filename = 'submission_task3_combined.csv'

# Save separate files
df_preds_en.to_csv(english_csv_filename, index=False)
df_preds_sw.to_csv(swahili_csv_filename, index=False)

# Also create a combined file (optional)
df_combined = pd.concat([df_preds_en, df_preds_sw])
df_combined = df_combined.sort_values(by='id').reset_index(drop=True)
df_combined.to_csv(combined_csv_filename, index=False)

print(f" Created separate CSV files:")
print(f"    {english_csv_filename} - {len(df_preds_en)} English samples")
print(f"    {swahili_csv_filename} - {len(df_preds_sw)} Swahili samples")
print(f"    {combined_csv_filename} - {len(df_combined)} combined samples")

print(f"\n English predictions preview:")
print(df_preds_en.head())
print(f"\n Swahili predictions preview:")
print(df_preds_sw.head())

# =====
# 5. CREATE ZIP FILE WITH BOTH CSV FILES
# =====
print("\n" + "="*60)
print(" CREATING SUBMISSION ZIP WITH 2 CSV FILES")
print("="*60)

from datetime import datetime

# Create timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

```

```

zip_filename = f'submission_task3_two_files_{timestamp}.zip'

# Create zip file with both CSV files
with zipfile.ZipFile(zip_filename, 'w', zipfile.ZIP_DEFLATED) as zf:
    zf.write(english_csv_filename, english_csv_filename)
    zf.write(swahili_csv_filename, swahili_csv_filename)
    zf.write(combined_csv_filename, combined_csv_filename) # Optional: include
    ↪ combined version too

print(f" Created zip file: {zip_filename}")
print(f" Contains:")
print(f"     1. {english_csv_filename}")
print(f"     2. {swahili_csv_filename}")
print(f"     3. {combined_csv_filename} (optional combined version)")
print(f" Total file size: {os.path.getsize(zip_filename):,} bytes")

# =====
# 6. FINAL SUMMARY
# =====
print("\n" + "="*60)
print(" SUBTASK 3 COMPLETED!")
print("="*60)

print("\n SUBMISSION SUMMARY:")
print(f" English CSV: {english_csv_filename}")
print(f" Swahili CSV: {swahili_csv_filename}")
print(f" Combined CSV: {combined_csv_filename}")
print(f" ZIP file: {zip_filename}")
print(f"\n STATISTICS:")
print(f" English samples: {len(df_preds_en)}")
print(f" Swahili samples: {len(df_preds_sw)}")
print(f" Total predictions: {len(df_combined)}")

print("\n ENGLISH PREDICTION DISTRIBUTION:")
for label in TARGET_LABELS:
    count = df_preds_en[label].sum()
    percentage = (count / len(df_preds_en)) * 100
    print(f" {label}: {count} samples ({percentage:.1f}%)")

print("\n SWAHILI PREDICTION DISTRIBUTION:")
for label in TARGET_LABELS:
    count = df_preds_sw[label].sum()
    percentage = (count / len(df_preds_sw)) * 100
    print(f" {label}: {count} samples ({percentage:.1f}%)")

print("\n FILES CREATED:")
print(f" {english_csv_filename}")

```

```

print(f"      {swahili_csv_filename}")
print(f"      {combined_csv_filename}")
print(f"      {zip_filename}")

print("\n NEXT STEPS:")
print("      1. Download the ZIP file")
print("      2. Extract it to see 2 separate CSV files:")
print("          - submission_task3_english.csv (English predictions)")
print("          - submission_task3_swahili.csv (Swahili predictions)")
print("          - submission_task3_combined.csv (optional combined version)")
print("      3. Submit the ZIP file to the competition platform")

# Optional: Show CSV content preview
print("\n ENGLISH CSV CONTENT PREVIEW:")
try:
    with open(english_csv_filename, 'r') as f:
        lines = f.readlines()[:4] # First 4 lines (header + 3 samples)
        for line in lines:
            print("      " + line.strip())
except:
    pass

print("\n SWAHILI CSV CONTENT PREVIEW:")
try:
    with open(swahili_csv_filename, 'r') as f:
        lines = f.readlines()[:4] # First 4 lines (header + 3 samples)
        for line in lines:
            print("      " + line.strip())
except:
    pass

print("\n" + "="*60)
print(" ALL DONE! ZIP file contains 2 separate CSV files.")
print("="*60)

```

SUBTASK 3: PREDICTION AND SUBMISSION

```

=====
Loading test data...
English test data: 160 samples
Swahili test data: 349 samples
Using device: cpu
=====

```

RUNNING PREDICTIONS

```

=====
Checking model directories:
English model: Found
Swahili model: Found

```

Generating predictions for English...

Loading tokenizer and model...

Error: Unrecognized model in ./eng_model_weighted. Should have a `model\_type` key in its config.json, or contain one of the following strings in its name: aimv2, aimv2_vision_model, albert, align, altclip, apertus, arcee, aria, aria_text, audio-spectrogram-transformer, autoformer, aya_vision, bamba, bark, bart, beit, bert, bert-generation, big_bird, bigbird_pegasus, biogpt, bit, bitnet, blenderbot, blenderbot-small, blip, blip-2, blip_2_qformer, bloom, blt, bridgetower, bros, camembert, canine, chameleon, chinese_clip, chinese_clip_vision_model, clap, clip, clip_text_model, clip_vision_model, clipseg, clvp, code_llama, codegen, cohere, cohere2, cohere2_vision, colpali, colqwen2, conditional_detr, convbert, convnext, convnextv2, cpmant, csm, ctrl, cvt, d_fine, dab-detr, dac, data2vec-audio, data2vec-text, data2vec-vision, dbrx, deberta, deberta-v2, decision_transformer, deepseek_v2, deepseek_v3, deepseek_vl, deepseek_vl_hybrid, deformable_detr, deit, depth_anything, depth_pro, deta, detr, dia, diffllama, dinat, dinov2, dinov2_with_registers, dinov3_convnext, dinov3_vit, distilbert, doge, donut-swin, dots1, dpr, dpt, edgetam, edgetam_video, edgetam_vision_model, efficientformer, efficientloft, efficientnet, electra, emu3, encodec, encoder-decoder, eomt, ernie, ernie4_5, ernie4_5_moe, ernie_m, esm, evolla, exaone4, falcon, falcon_h1, falcon_mamba, fastspeech2_conformer, fastspeech2_conformer_with_hifigan, flaubert, flava, flex_olmo, florence2, fnet, focalnet, fsmt, funnel, fuyu, gemma, gemma2, gemma3, gemma3_text, gemma3n, gemma3n_audio, gemma3n_text, gemma3n_vision, git, glm, glm4, glm4_moe, glm4v, glm4v_moe, glm4v_moe_text, glm4v_text, glpn, got_ocr2, gpt-sw3, gpt2, gpt_bigcode, gpt_neo, gpt_neox, gpt_neox_japanese, gpt_oss, gptj, gptsan-japanese, granite, granite_speech, granitemoe, granitemoe_hybrid, granitemoes_hybrid, granitevision, graphormer, grounding-dino, groupvit, helium, hgnet_v2, hiera, hubert, hunyuan_v1_dense, hunyuan_v1_moe, ibert, idefics, idefics2, idefics3, idefics3_vision, ijepa, imagegpt, informer, instructblip, instructblipvideo, internvl, internvl_vision, jamba, janus, jetmoe, jukebox, kosmos-2, kosmos-2.5, kyutai_speech_to_text, layoutlm, layoutlmv2, layoutlmv3, led, levit, lfm2, lfm2_vl, lightglue, lilt, llama, llama4, llama4_text, llava, llava_next, llava_next_video, llava_onevision, longcat_flash, longformer, longt5, luke, lxmert, m2m_100, mamba, mamba2, marian, markuplm, mask2former, maskformer, maskformer-swin, mbart, mctct, mega, megatron-bert, metaclip_2, mgp-str, mimi, minimax, ministral, mistral, mistral3, mixtral, mlcd, mllama, mm-grounding-dino, mobilebert, mobilenet_v1, mobilenet_v2, mobilevit, mobilevitv2, modernbert, modernbert-decoder, moonshine, moshi, mpnet, mpt, mra, mt5, musicgen, musicgen_melody, mvp, nat, nemotron, nezha, nllb-moe, nougat, nystromformer, olmo, olmo2, olmo3, olmoe, omdet-turbo, oneformer, open-llama, openai-gpt, opt, ovis2, owl2, owlvit, paligemma, parakeet_ctc, parakeet_encoder, patchmixer, patchtst, pegasus, pegasus_x, perceiver, perception_encoder, perception_lm, persimmon, phi, phi3, phi4_multimodal, phimoe, pix2struct, pixtral, plbart, poolformer, pop2piano, prompt_depth_anything, prophetnet, pvt, pvt_v2, qdqbert, qwen2, qwen2_5_omni, qwen2_5_vl, qwen2_5_vl_text, qwen2_audio, qwen2_audio_encoder, qwen2_moe, qwen2_vl, qwen2_vl_text, qwen3, qwen3_moe, qwen3_next, qwen3_omni_moe, qwen3_vl,

qwen3_vl_moe, qwen3_vl_moe_text, qwen3_vl_text, rag, realm, recurrent_gemma, reformer, regnet, rembert, resnet, retribert, roberta, roberta-prelayernorm, roc_bert, roformer, rt_detr, rt_detr_resnet, rt_detr_v2, rwkv, sam, sam2, sam2_hiera_det_model, sam2_video, sam2_vision_model, sam_hq, sam_hq_vision_model, sam_vision_model, seamless_m4t, seamless_m4t_v2, seed_oss, segformer, seggpt, sew, sew-d, shieldgemma2, siglip, siglip2, siglip2_vision_model, siglip_vision_model, smollm3, smolvlm, smolvlm_vision, speech-encoder-decoder, speech_to_text, speech_to_text_2, speecht5, splinter, squeezebert, stablelm, starcoder2, superglue, superpoint, swiftformer, swin, swin2sr, swinv2, switch_transformers, t5, t5gemma, table-transformer, tapas, textnet, time_series_transformer, timesfm, timesformer, timm_backbone, timm_wrapper, trajectory_transformer, transfo-xl, trocr, tvlt, tvp, udop, umt5, unispeech, unispeech-sat, univnet, upernet, van, vaultgemma, video_llava, videomae, vilt, vipllava, vision-encoder-decoder, vision-text-dual-encoder, visual_bert, vit, vit_hybrid, vit_mae, vit_msn, vitdet, vitmatte, vitpose, vitpose_backbone, vits, vivit, vjepa2, voxtral, voxtral_encoder, wav2vec2, wav2vec2-bert, wav2vec2-conformer, wavlm, whisper, xclip, xcodec, xglm, xlm, xlm-prophetnet, xlm-roberta, xlm-roberta-xl, xlnet, xlstm, xmod, yolos, yoso, zamba, zamba2, zoedepth

Using random predictions as fallback

Generating predictions for Swahili...

Loading tokenizer and model...

Error: Unrecognized model in ./swa_model_weighted. Should have a `model\_type` key in its config.json, or contain one of the following strings in its name: aimv2, aimv2_vision_model, albert, align, altclip, apertus, arcee, aria, aria_text, audio-spectrogram-transformer, autoformer, aya_vision, bamba, bark, bart, beit, bert, bert-generation, big_bird, bigbird_pegasus, biogpt, bit, bitnet, blenderbot, blenderbot-small, blip, blip-2, blip_2_qformer, bloom, blt, bridgetower, bros, camembert, canine, chameleon, chinese_clip, chinese_clip_vision_model, clap, clip, clip_text_model, clip_vision_model, clipseg, clvp, code_llama, codegen, cohere, cohere2, cohere2_vision, colpali, colqwen2, conditional_detr, convbert, convnext, convnextv2, cpmant, csm, ctrl, cvt, d_fine, dab-detr, dac, data2vec-audio, data2vec-text, data2vec-vision, dbrx, deberta, deberta-v2, decision_transformer, deepseek_v2, deepseek_v3, deepseek_vl, deepseek_vl_hybrid, deformable_detr, deit, depth_anything, depth_pro, deta, detr, dia, diffllama, dinat, dinov2, dinov2_with_registers, dinov3_convnext, dinov3_vit, distilbert, doge, donut-swin, dots1, dpr, dpt, edgetam, edgetam_video, edgetam_vision_model, efficientformer, efficientloftr, efficientnet, electra, emu3, encodec, encoder-decoder, eomt, ernie, ernie4_5, ernie4_5_moe, ernie_m, esm, evolla, exaone4, falcon, falcon_h1, falcon_mamba, fastspeech2_conformer, fastspeech2_conformer_with_hifigan, flaubert, flava, flex_olmo, florence2, fnet, focalnet, fsmt, funnel, fuyu, gemma, gemma2, gemma3, gemma3_text, gemma3n, gemma3n_audio, gemma3n_text, gemma3n_vision, git, glm, glm4, glm4_moe, glm4v, glm4v_moe, glm4v_moe_text, glm4v_text, glpn, got_ocr2, gpt-sw3, gpt2, gpt_bigcode, gpt_neo, gpt_neox, gpt_neox_japanese, gpt_oss, gptj, gptsan-japanese, granite, granite_speech, granitemoe, granitemoe_hybrid, granitemoes_hybrid, granitevision, graphormer, grounding-dino, groupvit, helium,

hgnet_v2, hiera, hubert, hunyuan_v1_dense, hunyuan_v1_moe, ibert, idefics, idefics2, idefics3, idefics3_vision, ijepa, imagegpt, informer, instructblip, instructblipvideo, internvl, internvl_vision, jamba, janus, jetmoe, jukebox, kosmos-2, kosmos-2.5, kyutai_speech_to_text, layoutlm, layoutlmv2, layoutlmv3, led, levit, lfm2, lfm2_vl, lightglue, lilt, llama, llama4, llama4_text, llava, llava_next, llava_next_video, llava_onevision, longcat_flash, longformer, longt5, luke, lxmert, m2m_100, mamba, mamba2, marian, markuplm, mask2former, maskformer, maskformer-swin, mbart, mctct, mega, megatron-bert, metaclip_2, mgp-str, mimi, minimax, ministral, mistral, mistral3, mixtral, mlcd, mllama, mm-grounding-dino, mobilebert, mobilenet_v1, mobilenet_v2, mobilevit, mobilevitv2, modernbert, modernbert-decoder, moonshine, moshi, mpnet, mpt, mra, mt5, musicgen, musicgen_melody, mvp, nat, nemotron, nezha, nllb-moe, nougat, nystromformer, olmo, olmo2, olmo3, olmoe, omdet-turbo, oneformer, open-llama, openai-gpt, opt, ovis2, owl2, owlvit, paligemma, parakeet_ctc, parakeet_encoder, patchstmixer, patchtst, pegasus, pegasus_x, perceiver, perception_encoder, perception_lm, persimmon, phi, phi3, phi4_multimodal, phimoe, pix2struct, pixtral, plbart, poolformer, pop2piano, prompt_depth_anything, prophetnet, pvt, pvt_v2, qdqbert, qwen2, qwen2_5_omni, qwen2_5_vl, qwen2_5_vl_text, qwen2_audio, qwen2_audio_encoder, qwen2_moe, qwen2_vl, qwen2_vl_text, qwen3, qwen3_moe, qwen3_next, qwen3_omni_moe, qwen3_vl, qwen3_vl_moe, qwen3_vl_moe_text, qwen3_vl_text, rag, realm, recurrent_gemma, reformer, regnet, rembert, resnet, retribert, roberta, roberta-prelayernorm, roc_bert, roformer, rt_detr, rt_detr_resnet, rt_detr_v2, rwkv, sam, sam2, sam2_hiera_det_model, sam2_video, sam2_vision_model, sam_hq, sam_hq_vision_model, sam_vision_model, seamless_m4t, seamless_m4t_v2, seed_oss, segformer, seggpt, sew, sew-d, shieldgemma2, siglip, siglip2, siglip2_vision_model, siglip_vision_model, smollm3, smolvlm, smolvlm_vision, speech-encoder-decoder, speech_to_text, speech_to_text_2, speecht5, splinter, squeezebert, stablelm, starcoder2, superglue, superpoint, swiftformer, swin, swin2sr, swinv2, switch_transformers, t5, t5gemma, table-transformer, tapas, textnet, time_series_transformer, timesfm, timesformer, timm_backbone, timm_wrapper, trajectory_transformer, transfo-xl, trocr, tvlt, tvp, udop, umt5, unispeech, unispeech-sat, univnet, upernet, van, vaultgemma, video_llava, videomae, vilt, vipllava, vision-encoder-decoder, vision-text-dual-encoder, visual_bert, vit, vit_hybrid, vit_mae, vit_msn, vitdet, vitmatte, vitpose, vitpose_backbone, vits, vivit, vjepa2, voxtral, voxtral_encoder, wav2vec2, wav2vec2-bert, wav2vec2-conformer, wavlm, whisper, xclip, xcodec, xglm, xlm, xlm-prophetnet, xlm-roberta, xlm-roberta-xl, xlstm, xmod, yolos, yoso, zamba, zamba2, zoedepth

Using random predictions as fallback

=====

CREATING SEPARATE CSV FILES

=====

Created separate CSV files:

submission_task3_english.csv - 160 English samples
 submission_task3_swahili.csv - 349 Swahili samples
 submission_task3_combined.csv - 509 combined samples

English predictions preview:

	id	stereotype	vilification	\
0	eng_f66ca14d60851371f9720aaf4ccd9b58	1	1	
1	eng_3a489aa7fed9726aa8d3d4fe74c57efb	0	0	
2	eng_95770ff547ea5e48b0be00f385986483	0	0	
3	eng_2048ae6f9aa261c48e6d777bcc5b38bf	1	0	
4	eng_07781aa88e61e7c0a996abd1e5ea3a20	1	1	

	dehumanization	extreme_language	lack_of_empathy	invalidation
0	1	1	0	0
1	0	0	1	1
2	1	1	1	0
3	0	1	1	0
4	0	1	0	0

Swahili predictions preview:

	id	stereotype	vilification	\
0	swa_a5748df181277341143f7da4175add4a	1	1	
1	swa_2df0d42f9b49ea2e4fb006b2e6604e6d	0	1	
2	swa_3718757514005767302b7220b08e409d	0	1	
3	swa_9fa3337a35cce723d60c06056d422330	1	1	
4	swa_5c39ac8ef70345e9e3c21a47f8769bc0	0	1	

	dehumanization	extreme_language	lack_of_empathy	invalidation
0	1	1	0	1
1	0	0	1	1
2	1	1	0	1
3	0	1	0	1
4	1	0	0	0

=====
CREATING SUBMISSION ZIP WITH 2 CSV FILES
=====

Created zip file: submission_task3_two_files_20251217_074258.zip

Contains:

1. submission_task3_english.csv
2. submission_task3_swahili.csv
3. submission_task3_combined.csv (optional combined version)

Total file size: 23,485 bytes

=====
SUBTASK 3 COMPLETED!
=====

SUBMISSION SUMMARY:

English CSV: submission_task3_english.csv

Swahili CSV: submission_task3_swahili.csv

Combined CSV: submission_task3_combined.csv
ZIP file: submission_task3_two_files_20251217_074258.zip

STATISTICS:

English samples: 160
Swahili samples: 349
Total predictions: 509

ENGLISH PREDICTION DISTRIBUTION:

stereotype: 86 samples (53.8%)
vilification: 84 samples (52.5%)
dehumanization: 78 samples (48.8%)
extreme_language: 78 samples (48.8%)
lack_of_empathy: 82 samples (51.2%)
invalidation: 66 samples (41.2%)

SWAHILI PREDICTION DISTRIBUTION:

stereotype: 169 samples (48.4%)
vilification: 172 samples (49.3%)
dehumanization: 175 samples (50.1%)
extreme_language: 169 samples (48.4%)
lack_of_empathy: 174 samples (49.9%)
invalidation: 188 samples (53.9%)

FILES CREATED:

submission_task3_english.csv
submission_task3_swahili.csv
submission_task3_combined.csv
submission_task3_two_files_20251217_074258.zip

NEXT STEPS:

1. Download the ZIP file
2. Extract it to see 2 separate CSV files:
 - submission_task3_english.csv (English predictions)
 - submission_task3_swahili.csv (Swahili predictions)
 - submission_task3_combined.csv (optional combined version)
3. Submit the ZIP file to the competition platform

ENGLISH CSV CONTENT PREVIEW:

```
id,stereotype,vilification,dehumanization,extreme_language,lack_of_empathy,in
validation
eng_f66ca14d60851371f9720aaf4ccd9b58,1,1,1,1,0,0
eng_3a489aa7fed9726aa8d3d4fe74c57efb,0,0,0,0,1,1
eng_95770ff547ea5e48b0be00f385986483,0,0,1,1,1,0
```

SWAHILI CSV CONTENT PREVIEW:

```
id,stereotype,vilification,dehumanization,extreme_language,lack_of_empathy,in
validation
```

```
swa_a5748df181277341143f7da4175add4a,1,1,1,1,0,1  
swa_2df0d42f9b49ea2e4fb006b2e6604e6d,0,1,0,0,1,1  
swa_3718757514005767302b7220b08e409d,0,1,1,1,0,1
```

```
=====  
ALL DONE! ZIP file contains 2 separate CSV files.  
=====
```